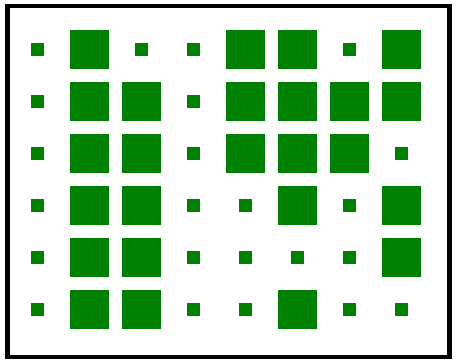


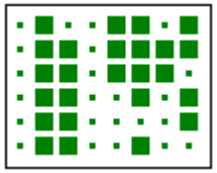


Class 2
Explore Learning and Language Evaluation

CS 330

Programming Languages

Team Concepts



- Couch potatoes article
  - Your thoughts

Vibe Coding Is Creating Braindead Coders



By Namanyay Goel
Sep 13 2025

https://nmn.gl/blog/vibe-coding-gambling

Confession: I've been using Claude Code to write all my code for me. And I think it's making me worse at the thing I've loved doing for twelve years.

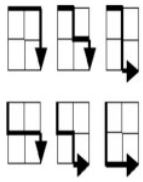
I can clearly see how AI coding is rewiring our brains – it makes developers crave instant gratification instead of deep understanding, and reduces us to **gamblers** who pull levers for the next *hit* of working code.

If this is happening to me, someone who learned to code in the pre-AI era, **what's it doing to junior developers who've never known anything else?**

Solving Hard Problems

At the age of 16, alone in my room in New Delhi, I was completely obsessed with this problem. Want to try solving this with me?

Starting in the top left corner of a 2 × 2 grid, and only being able to move to the right and down, there are exactly 6 routes to the bottom right corner.



How many such routes are there through a 20 × 20 grid?

It's from the site called "Project Euler", one of the greatest resource for programming challenges that combine math with your data structures and algorithms knowledge. This particular one is Problem 15 "Lattice Paths." Like all Project Euler problems, it can be stated simply enough.

LLM Roundtable

Vibe Coding Is Creating Braindead Coders



By Namanyay Goel
Sep 13 2025

https://nmn.gl/blog/vibe-coding-gambling

Confession: I've been using Claude Code to write all my code for me. And I think it's making me worse at the thing I've loved doing for twelve years.

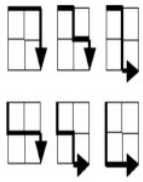
I can clearly see how AI coding is rewiring our brains – it makes developers crave instant gratification instead of deep understanding, and reduces us to **gamblers** who pull levers for the next *hit* of working code.

If this is happening to me, someone who learned to code in the pre-AI era, **what's it doing to junior developers who've never known anything else?**

Solving Hard Problems

At the age of 16, alone in my room in New Delhi, I was completely obsessed with this problem. Want to try solving this with me?

Starting in the top left corner of a 2 × 2 grid, and only being able to move to the right and down, there are exactly 6 routes to the bottom right corner.



How many such routes are there through a 20 × 20 grid?

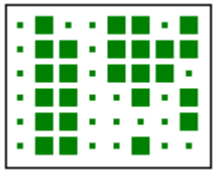
It's from the site called "Project Euler", one of the greatest resource for programming challenges that combine math with your data structures and algorithms knowledge. This particular one is Problem 15 "Lattice Paths." Like all Project Euler problems, it can be stated simply enough.

But **velocity isn't competency**. When you outsource thinking, you don't just lose practice — **you lose confidence**. That voice that used to whisper "I can figure this out" gets quieter every time you reach for AI instead.

Explore Learning

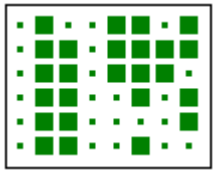
Advice from Prior Students

Advice 1/2

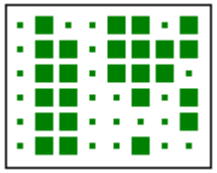


- Start early on compiler assignments
- Email professor when unclear
- Don’t be afraid to ask for help
- Take time to understand B-Minor
- Invest effort in the simple calculator lab
- Be patient, the hard work will payoff in the end
- The labs are where you really learn, start them as soon as you get them
- Make effort to grasp concept before jumping into code

Advice 2/2



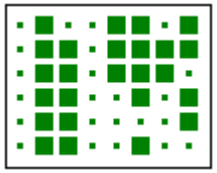
- There is class time given to work on labs when the professor is there – take advantage of it
- Read the textbook in depth
- Pay attention – all the information you need to be successful is in the book, slides, and lectures
- Make daily progress on labs
- Time management is crucial to succeed in this class
- Spend more times with readings and watch supplemental videos
- Start early – everything is manageable but time consuming



Programming Languages

Why study them conceptually?

Study Programming Languages?



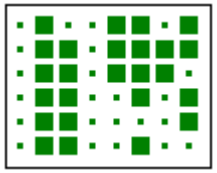
What is meant by “study programming languages?”

- Language Design
  - Syntax, safety, performance, paradigm
- Language Families
  - Imperative, OO, Functional, Logical
  - Compare and contrast
- Compiler
  - Lexing, parsing, verifying, generating, (linking, executing)

And a deep dive into the C language

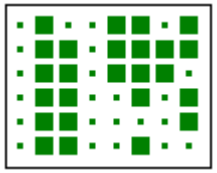
- Used to implement your compiler

Scope of This Course



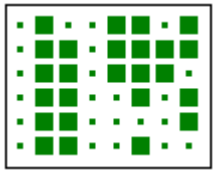
- We are focused on the design and implementation of programming languages
  - Explore different categories of languages
  - Understand issues with language design and implementation
  - Gain hands-on experience with several languages across language categories
- Other classes in the major cover
  - Writing efficient code
  - Testing and validation of programs
  - Software engineering practices and related tools

An Aside to Simplify Terminology



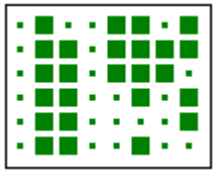
- This class is focused on **Programming Languages**
  - By “programming language” we are interested in general “Turing complete” languages
    - As opposed to syntax like HTML and XML
- To simplify discussion and slides I’ll use the terms “*language*” and “*languages*” in this class to mean “programming languages”
- This is not meant to imply *in any way* that programming languages are languages in the broader, human communication, sense

What's In It For You



- Broader language set allows you to (reasonably) express more ideas
- Have a basis for choosing appropriate language(s) for a given project
- Simplify the process of learning new programming languages and their features
- Explore the pros and cons of language design decisions
- Gain insights into the implementation of a language

But Wait, There's More



- Deeper insights with regard to any programming language you use
- Basis for considering language improvements and advancing computing in general
- The pride and war stories (lessons learned) creating one of the largest programs you have written to date

Who Needs Programs?











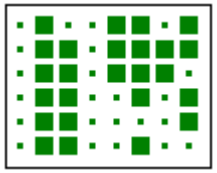




What are some differences between these computers?

How we might want to program them differently?

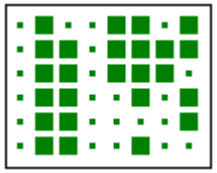
Domains of Programs (Example Langs)



- Scientific: Fortran, Algol 60, R
- Performant and Low-Level: C, C++
- Business (general purpose): COBOL, Java, Python, C#
- Artificial Intelligence (AI): Lisp, Prolog
- Web: JavaScript, Node.js, JEE
- Scripting: AWK, Perl, BASH, CSH

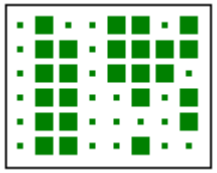
Thoughts on different needs of programs between these areas?

Domains of Programs (General Focus)



- Scientific: Numbers and calculations
- Performant and Low-Level: OS, Video editing, gaming
- Business: Many data types, interfaces, reporting
- Artificial Intelligence (AI): Flexible data and logic representation, including ability to adapt and recognize patterns
- Web: Diverse platforms and ease of use
- Scripting: Speed of development and text processing

TIOBE Index (as of September 2026)



September Headline: Is Julia going to re-enter the top 20?

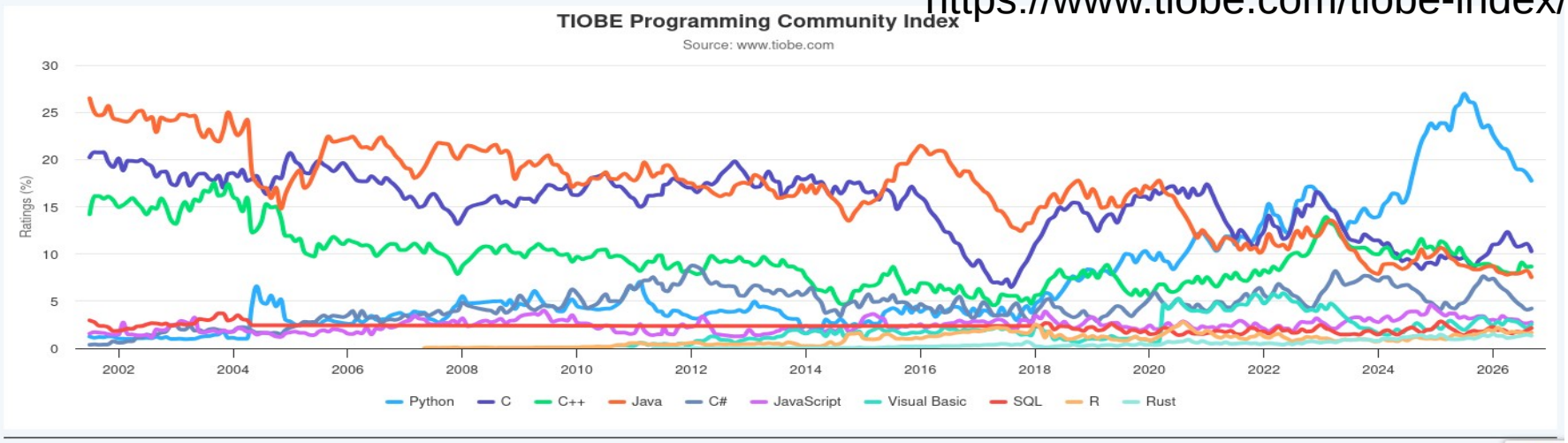
Summer of 2023, Julia entered the top 20 for the first time in its history. This month, it is close to making it back into that exclusive group. This month, MATLAB drops another two positions. Julia is one of the languages that has taken a share of MATLAB's territory. MATLAB, Julia, Python, and R all have strong roots in numerical computing, scientific computing, modeling, and data processing. Over the years, both Python and R have taken a significant share of MATLAB's market, but Julia has certainly claimed part as well.

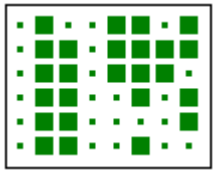
So far, Julia remains concentrated in relatively niche domains and has not yet managed to break through as a general-purpose programming language. It is fast, but there are faster languages. It is relatively easy to learn, but so are many alternatives. Julia has a strong and enthusiastic user community, although, somewhat surprisingly, it did not appear in last year's famous Stack Overflow survey of the most admired programming languages.

Top 10 remains remarkably stable, with some developments around position 20. Perl and Ruby have left the top 20, making room for Ada and Objective-C. Both languages have been around for decades, but are showing renewed strength in this month's ranking.

20	22	^	COBOL	0.78%	-0.13%

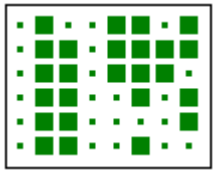
https://www.tiobe.com/tiobe-index/





Story Time...

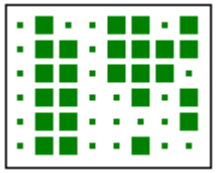
Historical Computers



- Another historical computer kit I’ve built
- The KENBAK-1 poses a challenge in programming
  - Input: Buttons
  - Output: Lights
- 256 bytes of memory
  - Program and data share this



Programming the KENBAK-1



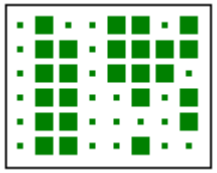
- Special Memory Locations

Octal Location	Purpose
000	A Register
001	B Register
002	X Register
003	P Register
200	Output Register
201	Overflow and Carry for the A Register
202	Overflow and Carry for the B Register
203	Overflow and Carry for the X Register
377	Input Register

- Instructions

ADD and SUB
LOAD and STORE
AND and OR
LNEG
JUMPS (BRANCHES)
SKIPS
SET BITS
SHIFTS and ROTATES
NOOP and HALT

A Simple Program (Counting by 1)



Load the program (all octal numbers)

	Location	Contents	Symbolic Instruction
	003	004	Initial value of P Reg.
LOOP	004	103	ADD B C=1
	005	001	
	006	134	STORE B OUTPUT
	007	200	
	010	344	JPD UNC LOOP
	011	004	

OUTPUT	200	xxx	

Steps:

1. Power On
2. Start
3. Stop
4. Clear
5. Enter 003
6. Set (Address)
7. Clear
8. Enter 004
9. Store (Memory)
10. Clear
11. Enter 103
12. Store
13. Clear
14. Enter 001
15. Store
16. Clear
17. Enter 134
18. Store
19. Clear
20. Enter 200
21. Store
22. Clear
23. Enter 344
24. Store
25. Clear
26. Enter 004
27. Store
28. Display (Address)

Step 27 concluded the entry process. Step 28 is a double check that the Address Register has the correct value which in this case should be 012, the next address. To check the entry of the data, which is a good habit to get into, or in general, to examine memory proceed with

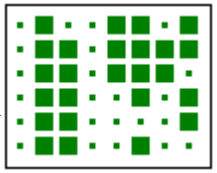
29. Clear
30. Enter 003
31. Set (Address)
32. Read (should be 004)
33. Read (should be 103)
34. Read (should be 001)
35. Read (should be 134)
36. Read (should be 200)
37. Read (should be 344)
38. Read (should be 004)

To run the program

39. Start

This program adds 1 to the B Register, puts the value of B out to location 200_8, and hence the Data Lamps, and recycles or loops. The binary counting process should be in evidence on the Data Lamps though the lamps will be operating too fast in the lower order positions to observe the action in detail.

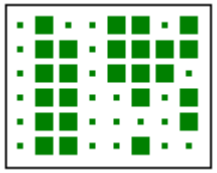
I Wanted to Write Dr. Nim for the KENBAK



- Simplified version of Nim
- Was produced as a game by ESR in the 1960s

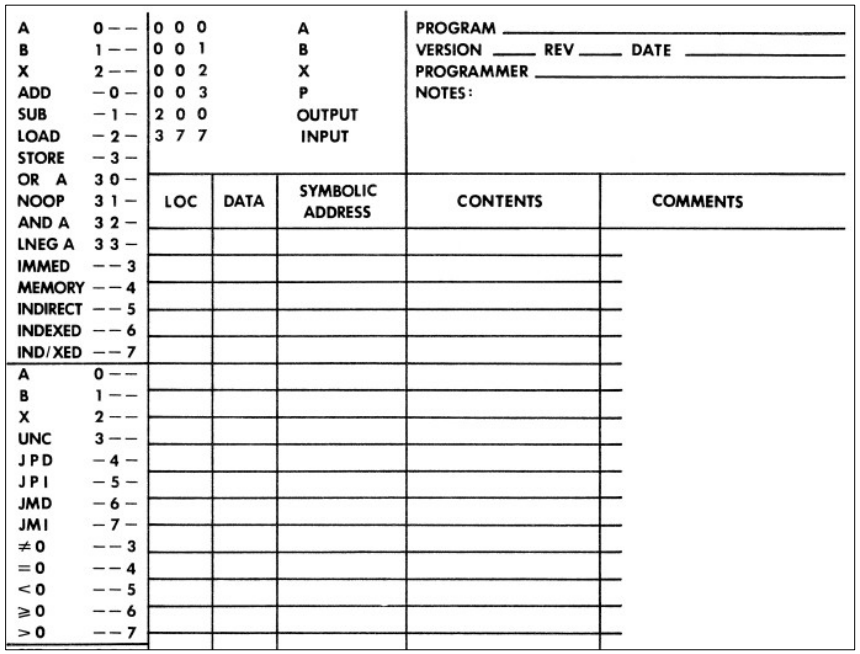
  https://boardgamegeek.com/video/81094/the-amazing-dr-nim/the-amazing-dr-nim-play-through
- Describing the pseudocode is easy:
  - Set marbles to 15
  - While there are marbles remaining
    - Human takes 1, 2 or 3
    - Dr. Nim takes 1, 2, or 3: looks to leave 1 marble or a multiple of 4 (+ 1) marbles. Otherwise, takes 1
  - Player who takes the last marble, loses

Implementation Using Op Codes

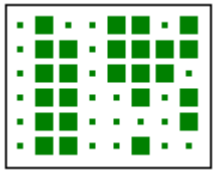


0000	0003	0077	0360
0000	0043	0224	0023
0000	0043	0000	0222
0004	0023	0114	0223
0123	0001	0002	0010
0017	0124	0024	0123
0134	0376	0001	0377
0376	0114	0134	0134
0241	0000	0376	0200
0134	0134	0241	0123
0200	0376	0104	0200
0123	0241	0002	0360
0000	0134	0134	0123
0134	0200	0200	0000
0377	0144	0044	0134
0024	0117	0126	0200
0200	0023	0000	0123
0323	0220	0344	0200
0360	0123	0013	0360
0034	0004	0023	0213
0200	0360	0220	0001
0023	0024	0123	0243
0220	0376	0003	0137
0123	0124	0360	0000
0001	0000	0344	0344
0360	0323	0133	0004
0000	0003	0023	
0024	0223	0220	
0377	0001	0123	
0323	0044	0006	

116 Bytes
- 3 bytes data
- 1 byte instruction pointer
- 113 bytes of instructions



Created a Language Aligned with Device



KBlang

Overview

KBlang provides a higher-level language for the microKENBAK-1's operating codes without adding complex abstractions. For the most part, the syntax parallels the microKENBAK-1 byte sequences using mnemonic keywords.

Statements may not be split across lines. Blank lines are ignored.

Numeric Literals

Numbers may be expressed in decimal, octal, or hexadecimal. A number without a leading 0 is interpreted as decimal, a number beginning with a leading 0 is interpreted as octal, and numbers beginning with 0x are interpreted as hexadecimal.

Most of the examples in this guide use octal values to align with the original KENBAK-1 programming guide which generally used octal numbers. However, KBlang allows equivalent decimal and hexadecimal numbers to be used anywhere an integer is expected.

Examples

Decimal values: 8, 15, 16, 127, 128, 255

Octal values: 010, 017, 020, 0177, 0200, 0377

Hexadecimal values: 0x8, 0xF, 0x10, 0x7F, 0x80, 0xFF

Keywords

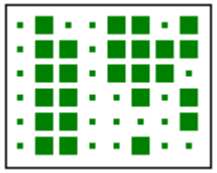
Keywords are not case sensitive. `LABEL`, `label`, and `Label` are equivalent.

The complete list of keywords are:

`ADD, ADDRESSIN, BITSHIFT, BYTES, FROM, GOTO, HALT, IF, ISZERO, LABEL, LEFT, LET, NOTZERO, RIGHT, SUBTRACT, SYSCALL, TO, VALUEIN`

https://github.com/DaveRead/microKENBAK-1-Compiler

Improved Semantics

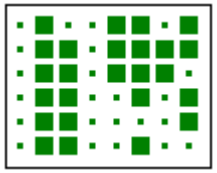


- Easier to read and write the program's logic

```
let b = valuein 0376
subtract a from b
memcopy b to 0376
bitshift b left 4
```

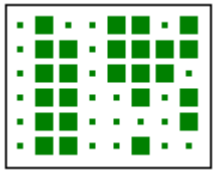
- Unrepresented syntax also adds value
  - Comments!
  - Statement labels for jumps
  - Keywords for reserved memory locations
    - A, B, X, Display, Input

Portion of the Program



```
# Light the INPUT LED
let a = 0220
let b = 1
syscall
# 3. Enter 1, 2 or 3 using the two right-most data inputs
# 4. Press Start to take the marbles
# Stop for input
halt
# Place input into A
let a = valuein input
# Force value in range 1-3
and 3
if a notzero goto input_okay
# Illegal input value, force to 1
let a = 1
label input_okay
# 5. The 4 left-most data LEDS display number of marbles
# Place number of marbles in B
let b = valuein 0376
```

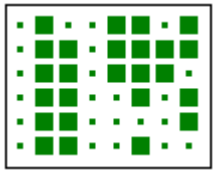
Can Create a More Readable Listing



Operation Codes for DrNim.kb1

Statement Number	Statement	Memory Location, Op Codes, and Values
1	label new_game	0004: (label new_game)
2	let b = 15	0004: 0123 (01 010 011) 0005: 0017 (00 001 111)
3	memcopy b to 0376	0006: 0134 (01 011 100) 0007: 0376 (11 111 110)
4	bitshift b left 4	0010: 0241 (10 100 001)
5	memcopy b to 0200	0011: 0134 (01 011 100) 0012: 0200 (10 000 000)
6	label player_turn	0013: (label player_turn)
7	let b = 0	0013: 0123 (01 010 011) 0014: 0000 (00 000 000)
8	memcopy b to 0377	0015: 0134 (01 011 100) 0016: 0377 (11 111 111)
9	let a = valuein display	0017: 0024 (00 010 100) 0020: 0200 (10 000 000)
10	and 0360	0021: 0323 (11 010 011) 0022: 0360 (11 110 000)
11	memcopy a to 0200	0023: 0034 (00 011 100) 0024: 0200 (10 000 000)

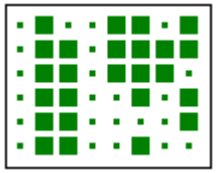
Some Takeaways



"Why would you want more than machine language?"

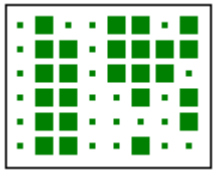
– John von Neumann

"The purpose of abstraction is not to be vague, but to create a new semantic level in which one can be absolutely precise."
– Edsger Dijkstra



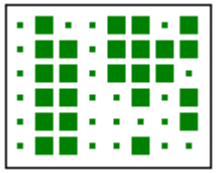
Language Evaluation

Core Language Evaluation Concepts



- Idea is to evaluate how effective the features of a language are at allowing programmers to create programs based on certain criteria
  - Gives us a basis to compare languages to each other
  - This is different from comparing how readable 2 programs written in the same language are

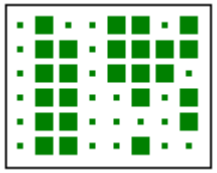
Common Criteria for Lang. Eval.



- Readability
- Writeability
- Reliability
- Total cost of operation (TCO)

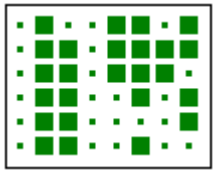
Do we need to consider the domain of the program being created?

Read/Writeability: Simplicity



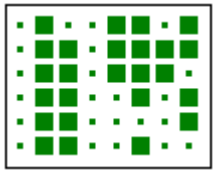
- Few constructs versus many features
- One way to do things versus many ways to achieve the same end
- One purpose for each operator versus overloading

Readability and Writeability: Orthogonal



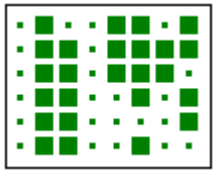
- Very orthogonal language would have few constructs with few combinations, and all would be legal and useful
- The more orthogonal a language, the more “common sense” seems to apply when using it versus memorizing exceptions to rules

Readability and Writeability: Data Types



- None, and the context of usage may lead to ambiguity
- Too few, and the true purpose of the data may be lost (using a number to indicate true/false)
- Too many, or non-orthogonal rules for how to use them, and arbitrary complexity abounds

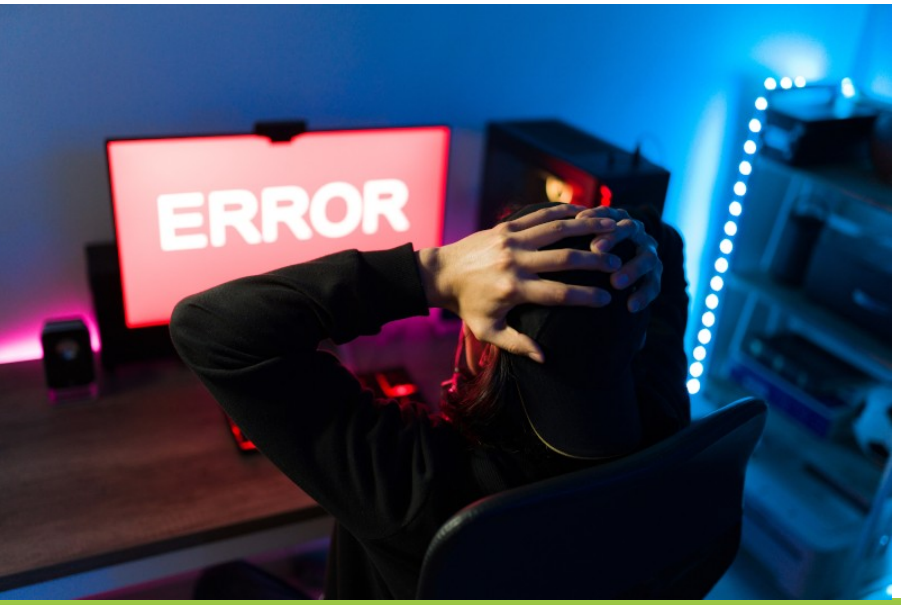
Read&Writeability: Syntax&Expression

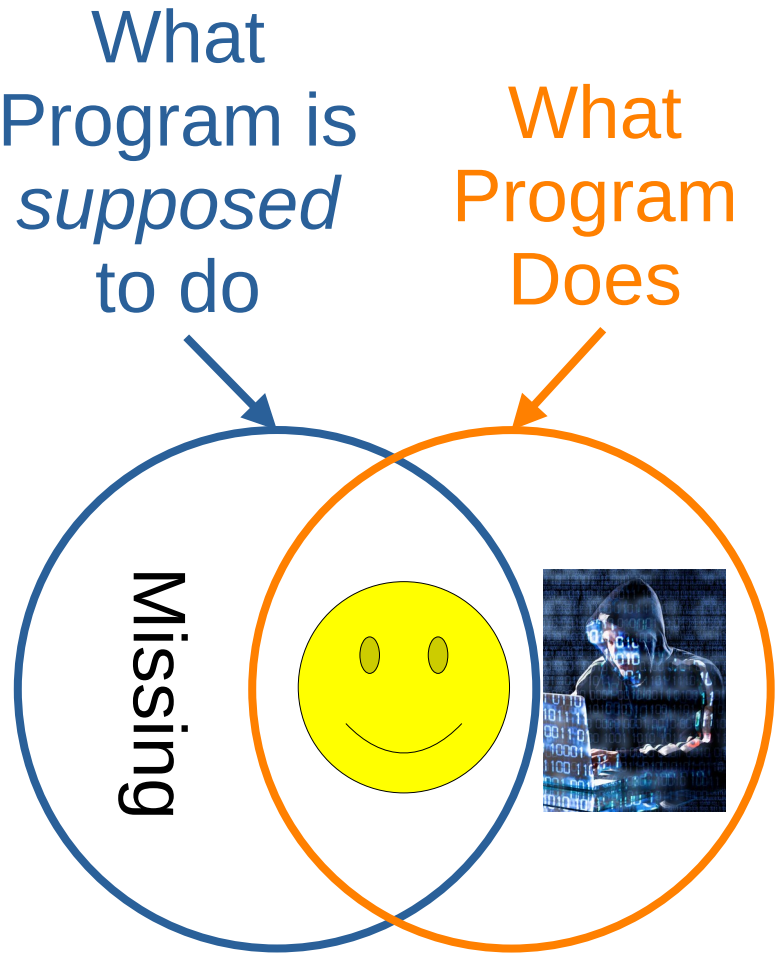


- Syntax
  - Keyword choice: clear, unambiguous?
  - Statement indication: visibility?
  - Block indication : visibility, structure?
- Expressivity
  - High-level operations (powerful operators)
  - Conveniences for common operations

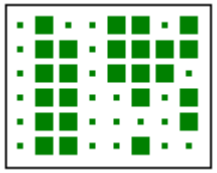
Reliability

The program does what it is supposed to do *AND* does not do anything else





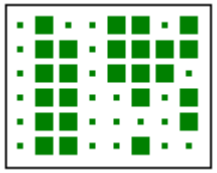
What Influences Reliability?



Key tenant: design the language to prevent families of errors (e.g., cause them to be caught by the interpreter or compiler)

- Type checking
- Exception handling
- Aliasing
- Readability and writeability

Influence TCO

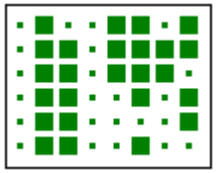


- Availability of trained talent
  - Training if insufficient
- Effort: Design, Development, Testing, Debug, Maintenance
- Runtime: Efficiency, Reliability

What influences these areas?

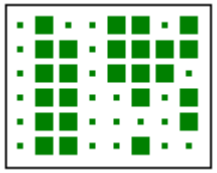
Where is most time spent?

Other Evaluation Criteria



- Can you think of other ways you'd want to evaluate a programming language?
- What about:
  - Case sensitivity for keywords and/or user defined names
  - Ability to abbreviate keywords (e.g. only required to supply the beginning up to it being unique)
- Does the language family change how you might evaluate?

Homework



- Sunday (9/13):
  - Journal entry for week 1
    - **2-1 Rule** (minimum 2 paragraphs, 1 quiz question)
- Monday (9/14):
  - Compilers book Chapters 1 and 2
    (Perusall annotations)



- Friday (9/18):
  - Compilers book Chapter 3
    (Perusall annotations)
  - LLM: AI Coding Assistants…
    (Perusall annotations)



